Tiny code (SQL) - Fine-Tuning Lightweight Language Models for BigQuery SQL Generation

Sanjay Kumar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SanjayKumar@sandiego.edu

Santa Das Karmakar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SDasKarmakar@sandiego.edu

Sethuraman Krishnasamy
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SKrishnasamy@sandiego.edu

Introduction

In the contemporary landscape of generative artificial intelligence, large-scale proprietary models such as GPT-5 and Gemini 3.0 Pro have established an extraordinarily high benchmark for code generation. However, for the modern enterprise, these "frontier" models present formidable barriers to entry that extend beyond mere subscription costs. These include prohibitive per-token inference expenses, high operational latency, and most critically the existential security risk associated with transmitting sensitive internal database schemas to external third party cloud environments. Furthermore, while general-purpose Small Language Models (SLMs) in the 1B–7B parameter range offer a more portable alternative, they frequently lack the "idiomatic reasoning" and dialect-specific nuance required to navigate complex Google BigQuery joins or adhere to strict organizational coding standards. This leads to the "hallucination" phenomenon, where a model produces syntactically plausible but logically flawed code—such as incorrect JOIN keys or a failure to respect schema constraints—errors that a larger, more cognitively dense model would typically avoid.

This project investigates the "efficiency gap" within domain-specific artificial intelligence, specifically evaluating whether a sub-7B parameter language model can be mathematically and architecturally optimized to match the SQL proficiency of ultra-large-scale architectures. By fine tuning a lightweight model exclusively for the BigQuery dialect, this project seeks to bridge the "Small Model Gap." The objective is to maintain a memory footprint lean enough for serverless environments, such as GCP Cloud Functions, and edge-tier deployment, where

traditional multi hundred-billion parameter models cannot operate.

This project is vital for enabling Data Sovereignty and the model is designed to live where the data lives. By specializing a model for in-house use, organizations can maintain their sensitive.

INFORMATION_SCHEMA—the metadata blueprint of their entire data warehouse— within their own private cloud perimeter. In a live production environment, the system is designed to simulate a secure "metadata injection" pipeline. Instead of transmitting data to external APIs, the model locally ingests the company's internal database catalog and private schema definitions. This allows for context-aware query generation that understands the specific nested fields, partitioned tables, and complex relationships unique to that enterprise. This architectural choice virtually eliminates data leakage risks while simultaneously slashing inference costs by an estimated 90%. Beyond security, this specialization enables "Instant SQL" by achieving sub-50ms per token latency, effectively removing the "thought-to-code" bottleneck and facilitating real-time developer productivity across a high velocity IT enterprise.

The primary end users of this AI product are Data Analysts, Data Engineers, and Software Engineers operating within Google Cloud Platform (GCP) environments who require instant, secure and accurate SQL generation. These professionals require a tool that can translate complex business logic from natural language into production ready SQL without the friction of manual coding or external API calls. The vision for the final product is an integrated, quantized, "edge-ready" model—packaged in high performance formats like GGUF or EXL2—that functions within a Streamlit dashboard or a localized browser extension. By targeting an execution accuracy exceeding 70% on the rigorous Spider SQL benchmark, the system aims to provide a reliable, "self-correcting" assistant for BigQuery synthesis. This allows engineers to integrate automated SQL generation directly into internal command-line interface (CLI) tools or secure, air-gapped local applications, ensuring that intelligence is available even in restricted networking environments.

The training strategy departs from the "more is better" data philosophy, instead utilizing a high-density corpus derived from The Stack v1 (BigCode Project), filtered with surgical precision for BigQuery-specific SQL. A refined subset (filtered specifically for BigQuery syntax) of the Stack v1 (BigCode Project) which consists of approximately 250,000 high-quality SQL files

(~3GB of data) is used for training. To ensure the model internalizes only the highest quality "idiomatic" patterns, the data is restricted to repositories with at least 50 stars and a comment-to-code density of 10%–30%. This heuristic ensures the model learns not just the syntax, but the relationship between human documentation and logical implementation. To bridge the gap in complex reasoning, we To bridge the gap in complex reasoning, Gemini 3.0 Pro is used as a "teacher" to generate 1,000 high complexity triplets consisting of $[Instruction + Schema] \rightarrow [Optimized\ Query]$. This pedagogical framework is designed to teach the model how to interpret complex database metadata, such as nested fields and partitioned tables.

The core hypothesis of this project is that high-density, domain-specific data, when combined with "Efficiency-First" engineering, can enable a 3B parameter model to match the functional utility of a 175B parameter generalist model for specialized coding tasks. This project relies specifically on two cutting-edge techniques: 4-bit QLoRA (Quantized Low-Rank Adaptation), which allows for high-fidelity fine-tuning on consumer-grade hardware, and Flash Attention 2, an IO-aware attention mechanism that accelerates training and reduces memory overhead. By dedicating the model's entire "parameter budget" to SQL-centric reasoning, we aim to deliver a "Vertical AI" specialist engineered to operate within the 60-second "cold start" constraints of serverless cloud functions. This optimization ensures the model operates within the 60-second 'cold start' window typical of serverless cloud functions, providing a high-velocity, secure alternative to the high-latency, resource heavy, general-purpose LLMs currently prevalent in enterprise AI.

Data Summary

The dataset employed for this project is the synthetic_text_to_sql corpus provided by Gretel AI and hosted on Hugging Face. It comprises 50,000 training samples (a random subset of the full training split) and 5,000 test samples, each consisting of a natural language user request, an accompanying database schema, a corresponding SQL query, and a step-by-step explanation of the query logic. In plain terms, the primary variables are: (1) the user's question expressed in everyday language, (2) the structural description of the tables and columns (schema context), (3) the executable SQL statement that answers the question, (4) a human-readable explanation of how the SQL works, (5) a categorical label indicating the overall complexity of the SQL (simple, moderate, or complex), and (6) a categorical label describing the type of SQL task being performed (for example, data retrieval,

aggregation, filtering, or joins). These elements were extracted directly into a pandas DataFrame for exploratory analysis.

No missing values or null entries were present in any column, as the dataset is fully synthetic and intentionally curated for completeness. The only data-quality adjustments required were dialect-specific normalizations to ensure compatibility with Google BigQuery: all SQL statements were automatically transpiled to the BigQuery dialect using the sqlglot library, backticks were enforced for identifier quoting, and any generic CAST operations were replaced with SAFE_CAST to prevent runtime failures on unexpected data types. In addition, a controlled augmentation pipeline introduced realistic noise—random distractor tables in 20 % of examples, sample-row grounding hints, and stylistic variations in question phrasing—to simulate production schema drift and diverse user writing styles.

These preprocessing steps were deterministic and reversible, so in a fully developed machine-learning pipeline the same transformations would be encapsulated in a reusable data-validation and feature-engineering service (e.g., a Cloud Dataflow job) that runs continuously on incoming real-world query logs, automatically logging any schema mismatches or length outliers for later retraining.

All variables are directly aligned with the project goal of producing an accurate BigQuery Text-to-SQL assistant. The natural-language prompt and schema serve as the model inputs, while the SQL statement and explanation constitute the target output; the complexity and task-type labels provide implicit difficulty signals that help the model learn progressive reasoning patterns.

Exploratory visualizations revealed a healthy diversity that supports robust generalization: the complexity distribution is well-balanced (approximately one-third simple, one-third moderate, and one-third complex), the task-type distribution covers the most common BigQuery operations without severe class imbalance, and the top SQL keywords (SELECT, FROM, WHERE, GROUP BY, JOIN, etc.) appear at frequencies that mirror real-world analytical workloads. Prompt and SQL character-length distributions are right-skewed but contain no extreme outliers, confirming that the chosen maximum sequence length of 64 tokens will accommodate the vast majority of examples without excessive truncation. These patterns collectively indicate that every variable will be useful to the model; no feature appears redundant or uninformative.

Because the input data are primarily free-text rather than numeric features, traditional Pearson correlations between variables are not directly

applicable. However, a modest positive relationship exists between prompt length and SQL length (longer natural-language requests tend to require correspondingly longer queries), and SQL complexity correlates loosely with the presence of advanced keywords such as JOIN, GROUP BY, and CASE. These relationships informed the modeling pipeline in two practical ways: gradient-accumulation steps were set to 4 to stabilize learning on longer sequences, and LoRA target modules were chosen to emphasize attention over the schema and prompt tokens. In production, the same length and complexity statistics would be monitored via a dashboard; any drift toward longer or more complex queries would trigger an automatic increase in max_length or a second fine-tuning stage on the updated distribution, ensuring the model remains performant as real user traffic evolves.

Literature Review

Translating natural language queries into executable SQL statements (Text-to-SQL or NL2SQL) has long been pursued to lower barriers to relational database access for non-expert users. Early approaches relied on rule-based parsers or traditional machine learning techniques. For example, Zhong et al. (2017) introduced Seq2SQL, a reinforcement-learning-based seq2seq model trained to generate structured queries directly from natural language on benchmarks such as WikiSQL. Subsequent pre-large-language-model (PLM) methods, such as RESDSQL, employed encoder-decoder architectures with schema-linking techniques to handle complex multi-table joins and achieve strong results on the Spider benchmark with lower inference costs than pure LLM prompting.

With the rise of large language models (LLMs), two dominant paradigms emerged: in-context prompting and supervised fine-tuning. Prompting methods (e.g., DIN-SQL) decompose the task into sub-problems and provide few-shot examples or schema context within the input prompt, but they suffer from high token consumption and latency on complex queries. Surveys such as Liu et al. (2025) and Hong et al. (2025) document that, when labeled training data are available, supervised fine-tuning consistently outperforms prompting by adapting model parameters to better capture schema grounding, dialect-specific syntax, and reasoning patterns required for accurate SQL generation.

To make fine-tuning computationally tractable for billion-parameter models, parameter-efficient fine-tuning (PEFT) techniques were developed. Hu et al. (2021) introduced **Low-Rank Adaptation (LoRA)**, which freezes the pre-trained weights of a Transformer and injects

trainable low-rank decomposition matrices (rank $r \ll d$) into each layer. Compared with full fine-tuning of GPT-3 175B using Adam, LoRA reduces the number of trainable parameters by up to 10,000× and GPU memory requirements by 3× while matching or exceeding full fine-tuning performance on tasks including GPT-2 and GPT-3 code generation. Dettmers et al. (2023) extended this to **QLoRA** by combining 4-bit quantization (NF4) with LoRA, enabling fine-tuning of 65B-parameter models on a single 48 GB GPU without sacrificing downstream accuracy.

These PEFT methods have proven particularly well-suited to Text-to-SQL because the task benefits from domain-specific adaptation (schema understanding, dialect rules, and value grounding) while requiring far less data and compute than full retraining. Google's official Gemma fine-tuning documentation (2026) explicitly demonstrates this workflow: the philschmid/gretel-synthetic-text-to-sql dataset (a high-quality synthetic corpus of >100k natural-language-to-SQL pairs with schema, reasoning traces, and SQL outputs) is formatted into conversational templates and used to train Gemma models via QLoRA + Hugging Face TRL's SFTTrainer. The guide highlights that this approach enables production-ready SQL generation on modest hardware while preserving the base model's broad knowledge.

Multiple independent projects have applied the same LoRA/QLoRA + Gemma + Gretel combination. Community models on Hugging Face include:

- parole-study-viper/gemma3-text-to-sql — a LoRA adapter (rank 16, target modules covering all attention and MLP layers) on Gemma-3-27B-it, trained on synthetic and curated text-to-SQL data, reported to generate clean, schema-aware queries with joins, aggregations, and filters.
- litert-community/gemma3-1b-ft-text-to-sql and Yuk050/gemma-3-1b-text-to-sql-model — both QLoRA-fine-tuned on the Gretel synthetic dataset, explicitly created for efficient text-to-SQL inference on resource-constrained systems.

Commercial efforts follow a parallel trajectory. Defog's SQLCoder family (7B–70B variants) was built by fine-tuning CodeLlama on large text-to-SQL corpora; subsequent work has further improved these models with LoRA adapters and reinforcement learning from execution feedback, raising exact-match scores from 31.49 % to 49.83 % on benchmarks while reducing syntactic errors. These models have been shown to

outperform GPT-3.5 and approach GPT-4 on domain-specific SQL tasks.

Collectively, these surveys, foundational PEFT papers, official framework guides, open-source adapters, and commercial deployments demonstrate that LoRA/QLoRA-based fine-tuning of instruction-tuned LLMs (particularly Gemma-series models) on synthetic text-to-SQL datasets has become a mature, widely validated method for building high-accuracy, dialect-aware SQL generators. The approach addresses the core challenges of schema linking and query complexity while remaining feasible on single-GPU infrastructure — precisely the characteristics that make it appropriate for applied Text-to-SQL projects in 2025–2026.